

[Github Repository](#)

مهدی جمال خواه

Last Commit SHA: 3284aa8f0c9bf260937e2a637c5dac0b8ca304c5

810100111

بخش اول

نتیجه اجرای Pitest در شکل زیر نشان داده شده است:

```
=====
- Statistics
=====
>> Line Coverage (for mutated classes only): 50/50 (100%)
>> Generated 29 mutations Killed 28 (97%)
>> Mutations with no coverage 0. Test strength 97%
>> Ran 34 tests (1.17 tests per mutation)
```

شکل ۱.۱: خروجی Pitest در ترمینال

بر اساس شکل بالا:

- تعداد mutant-های ساخته شده: ۲۹ تا
- تعداد mutant-های کشته شده توسط تست‌های ما: ۲۸ تا
- تعداد mutant-های زنده مانده بعد از اجرای تست‌ها: یکی

گزارش Pit Test

در اینجا mutator-های زیر فعال بوده‌اند و بسته به اینکه روی کد قابل اعمال بوده‌اند یا نه از آن‌ها استفاده شده است:

Active mutators

- CONDITIONALS_BOUNDARY
- EMPTY_RETURNS
- FALSE_RETURNS
- INCREMENTS
- INVERT_NEGS
- MATH
- NEGATE_CONDITIONALS
- NULL_RETURNS
- PRIMITIVE_RETURNS
- TRUE_RETURNS
- VOID_METHOD_CALLS

شکل ۱.۲: Active mutators

Pit Test Coverage Report

Package Summary

domain

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
2	100% 50/50	97% 28/29	97% 28/29

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
Transaction.java	100% 9/9	100% 4/4	100% 4/4
TransactionEngine.java	100% 41/41	96% 24/25	96% 24/25

شکل ۱.۳: Pit Test Coverage Report

همان طور که در شکل ۱.۳ پیداست تمامی خطوط کد پوشش داده شده‌اند و از ۲۹ تا mutant ایجاد شده ۲۸ تای آن‌ها کشته شده‌اند اما آن یکی که زنده مانده است به این خاطر است که از نوع equivalent mutant می‌باشد و جزئیات آن را در شکل‌های بعدی بهتر متوجه خواهیم شد.

Transaction.java

```
1 package domain;
2
3 import lombok.Getter;
4 import lombok.Setter;
5
6 @Getter
7 @Setter
8 public class Transaction {
9     int transactionId;
10    int accountId;
11    int amount;
12    boolean isDebit;
13
14    @Override
15    public boolean equals(Object obj) {
16        if (obj instanceof Transaction transaction) {
17            return transactionId == transaction.transactionId;
18        }
19        return false;
20    }
21 }
```

Mutations

```
16 1. negated conditional → KILLED
17 1. replaced boolean return with true for domain/Transaction::equals → KILLED
   2. negated conditional → KILLED
19 1. replaced boolean return with true for domain/Transaction::equals → KILLED
```

شکل ۱.۴: Transaction class report

برای کلاس Transaction همان گونه که در شکل ۳.۱ نشان داده شده است ۴ تا mutant ساخته شده است که دو تای آن از نوع negated conditional و دو تای دیگر عوض کردن جمله return با true بوده است. که هر ۴ تای آنها کشته شده‌اند. البته همان گونه که در شکل ۱.۲ نوشته شده است مدل‌های دیگری هم فعال بوده است، اما برای این تکه کد قابل اعمال نبوده‌اند.

TransactionEngine.java

```
1 package domain;
2
3 import java.util.ArrayList;
4
5 public class TransactionEngine {
6     ArrayList<Transaction> transactionHistory;
7     int THRESHOLD = 1000;
8
9     public TransactionEngine() {
10         transactionHistory = new ArrayList<>();
11     }
12
13     int getAverageTransactionAmountByAccount(int accountId) {
14         var totalAmount = 0;
15         var count = 0;
16
17         for (Transaction txn : transactionHistory) {
18             if (txn.accountId == accountId) {
19                 totalAmount += txn.amount;
20                 count++;
21             }
22         }
23
24         if (count == 0) {
25             return 0;
26         }
27
28         return totalAmount / count;
29     }
30
31     int getTransactionPatternAboveThreshold(int threshold) {
32         if (transactionHistory.isEmpty()) {
33             return 0;
34         }
35
36         var diff = 0;
37         var previous = transactionHistory.getFirst();
38
39         for (Transaction txn : transactionHistory) {
40             if (txn.transactionId == previous.transactionId) {
41                 continue;
42             }
```

شکل ۱.۴: TransactionEngine class report - Part I

```

43
44 2      if (txn.amount <= threshold) {
45          continue;
46      }
47
48 1      if (diff == 0) {
49 1          diff = txn.amount - previous.amount;
50          previous = txn;
51 2      } else if (diff != txn.amount - previous.amount) {
52          return 0;
53      }
54  }
55
56 1      return diff;
57  }
58
59      int detectFraudulentTransaction(Transaction txn) {
60          var averageAmount = getAverageTransactionAmountByAccount(txn.accountId);
61
62 4      if (txn.isDebit && txn.amount > 2 * averageAmount) {
63 3          return txn.amount - 2 * averageAmount; // Excessive debit, marked as suspicious
64      }
65
66          return 0;
67      }
68
69      public int addTransactionAndDetectFraud(Transaction txn) {
70 1          if (transactionHistory.contains(txn)) {
71              return 0;
72          }
73
74          var fraudScore = detectFraudulentTransaction(txn);
75 1          if (fraudScore == 0) {
76              fraudScore = getTransactionPatternAboveThreshold(THRESHOLD);
77          }
78
79          transactionHistory.add(txn);
80 1          return fraudScore;
81      }
82  }

```

شکل ۱.۵: TransactionEngine class report - Part II

Mutations

```

18 1. negated conditional → KILLED
19 1. Replaced integer addition with subtraction → KILLED
20 1. Changed increment from 1 to -1 → KILLED
24 1. negated conditional → KILLED
28 1. Replaced integer division with multiplication → KILLED
    2. replaced int return with 0 for domain/TransactionEngine::getAverageTransactionAmountByAccount → KILLED
32 1. negated conditional → KILLED
40 1. negated conditional → KILLED
44 1. negated conditional → KILLED
    2. changed conditional boundary → KILLED
48 1. negated conditional → KILLED
49 1. Replaced integer subtraction with addition → KILLED
51 1. negated conditional → KILLED
    2. Replaced integer subtraction with addition → KILLED
56 1. replaced int return with 0 for domain/TransactionEngine::getTransactionPatternAboveThreshold → KILLED
    1. Replaced integer multiplication with division → KILLED
62 2. negated conditional → KILLED
    3. negated conditional → KILLED
    4. changed conditional boundary → SURVIVED
63 1. Replaced integer subtraction with addition → KILLED
    2. replaced int return with 0 for domain/TransactionEngine::detectFraudulentTransaction → KILLED
    3. Replaced integer multiplication with division → KILLED
70 1. negated conditional → KILLED
75 1. negated conditional → KILLED
80 1. replaced int return with 0 for domain/TransactionEngine::addTransactionAndDetectFraud → KILLED

```

شکل ۱.۶: TransactionEngine class report - Part III

در این کلاس یک mutant زنده مانده است و آن از نوع changed conditional boundary بوده است که روی خط ۶۲ اجرا شده است. به طور دقیق تر کد تابع شامل این خط را در حالت اصلی و تغییر یافته بررسی می‌کنیم:

حالت اصلی:

```
int detectFraudulentTransaction(Transaction txn) {
    var averageAmount = getAverageTransactionAmountByAccount(txn.accountId);
    if (txn.isDebit && txn.amount > 2 * averageAmount) {
        return txn.amount - 2 * averageAmount;
    }
    return 0;
}
```

بعد از اعمال mutation:

```
int detectFraudulentTransaction(Transaction txn) {
    var averageAmount = getAverageTransactionAmountByAccount(txn.accountId);
    if (txn.isDebit && txn.amount >= 2 * averageAmount) {
        return txn.amount - 2 * averageAmount;
    }
    return 0;
}
```

همان طور که می‌بینید رفتار کد فقط زمانی تغییر می‌کند که isDebit true باشد و amount برابر با $averageAmount * 2$ باشد در این صورت به اشتباه وارد if می‌شویم. اما نکته این است که در این حالت درون if نیز مقدار صفر برمی‌گرداند. یعنی همان مقداری که کد اصلی به ازای این ورودی برمی‌گرداند. در نتیجه چنین mutant-ای هیچ وقت قابل کشته شدن نیست و به اصطلاح equivalent mutant می‌باشد.

تاثیر Mutation Coverage بالا در میزان خطر Refactoring:

Refactoring ساختار درونی کد را بهبود می‌دهد اما رفتار خروجی کد را نباید تغییر ندهد. مثلاً ما می‌خواهیم یک bad smell را از بین ببریم اما نباید به گونه‌ای آن را تغییر دهیم که دیگر خروجی مورد انتظار ما را نداشته باشد.

وقتی refactoring روی production code انجام می‌شود تست‌های ما می‌توانند برای اطمینان از تغییر نکردن رفتار خروجی کد استفاده شوند. البته منظور اطمینان صد درصدی نیست و بلکه یک سطحی از اطمینان بسته به اینکه از چه coverage criteria-ای استفاده کرده باشیم، منظور است. در این راستا mutation testing می‌تواند خوب عمل کند زیرا mutant-ها معمولاً رفتارهای اشتباه شایع در تغییر کد را شامل می‌شوند. و وقتی mutation coverage بالاست یعنی انتظار داریم که اگر چنین تغییرات اشتباهی در کد رخ داد تستی fail شود و جلوی ما را بگیرد.

اما وقتی که refactoring روی test code انجام می‌شود دیگر چنین safety net-ای وجود ندارد. اما استفاده از معیار mutation coverage می‌تواند یک safety net باشد. یعنی بعد از refactoring باید مقدار

این معیار تغییر نکند. تا مطمئن باشیم رفتار تست تغییر نکرده باشد. دوباره منظور اطمینان صد درصدی نیست و یک حد خوب از اطمینان را در نظر داریم. ([منبع](#))

بخش دوم

برای اینکار فایل buildAndTest.yml را در پوشه github/workflows قرار می‌دهیم.

```
# buildAndTest.yml
name: Build and Run Tests

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up JDK 21
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'oracle'
          cache: 'maven'

      - name: Run Mizdooni tests
        run: cd Mizdooni && mvn test -B

      - name: Run Fesadyab tests
        run: cd Fesadyab && mvn test -B
```

این فایل تست‌های هر دو پروژه را قبل از push کردن اجرا می‌کند و اگر تستی fail شود، push انجام نخواهد شد. در شکل زیر یک نتیجه اجرای این workflow در github نشان داده شده است.

MehdiJmlkh / Software-Testing-Course-Projects

<> Code

Issues

Pull requests

Actions

Projects

Security

Insights

Settings

← Build and Run Tests

✓

Add build and run tests workflow #1

Summary

Jobs

Run details

Usage

Workflow file

build

succeeded now in 1m 0s

> ✓ Set up job

> ✓ Checkout code

> ✓ Set up JDK 21

> ✓ Run Mizdooni tests

> ✓ Run Fesadyab tests

> ✓ Post Set up JDK 21

> ✓ Post Checkout code

> ✓ Complete job

شکل ۲.۱